

Loop Optimization

Chapter 18

Loop

- **Why:** Loops are **pervasive** in computer programs, and **a great proportion of the execution time** of a typical program is spent in one loop or another.
 - We want techniques to improve loops!
- **Loop** (in Webster's Dictionary): a series of instructions that is repeated until a terminating condition is reached.
- A **loop** in a control-flow graph is a set of nodes **S** including a header node **h** with the following properties:
 - From any node in **S** there is a **path** of directed edges leading to **h**.
 - There is a **path** of directed edges from **h** to any node in **S**.
 - There is no **edge** from any node outside **S** to any node in **S** other than **h**.

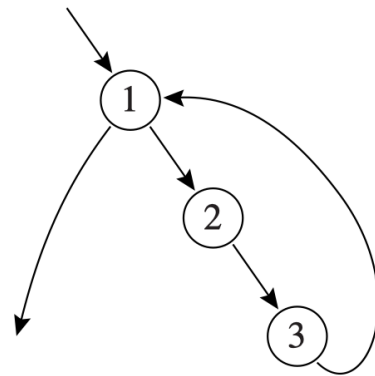
Loop - Example

- A **loop entry** node is one with predecessor outside the loop.
- A **loop exit** node is one with a successor outside the loop.

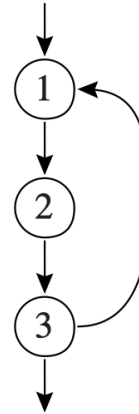
$h \rightarrow S$

$S \rightarrow h$

no other node $\rightarrow S$

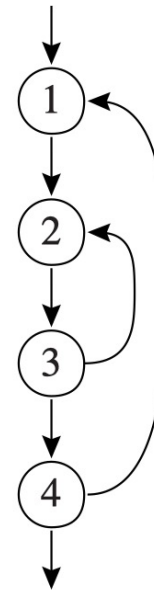
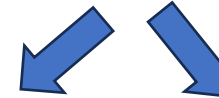


(a)

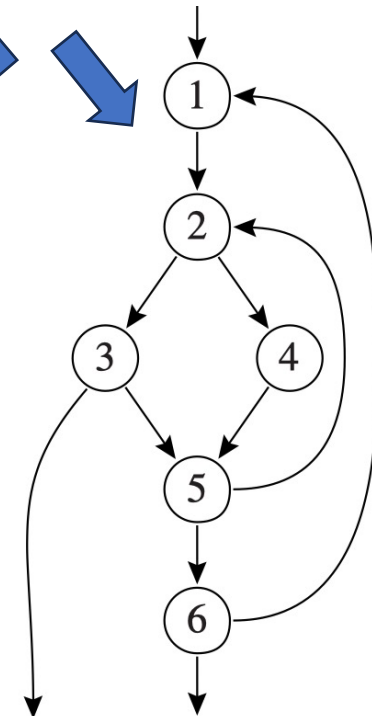


(b)

nested loops

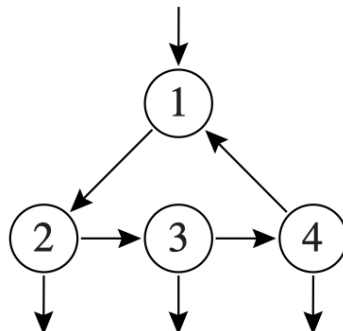


(e)

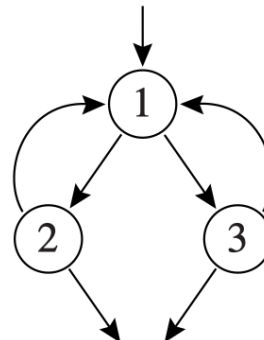


(f)

only **one** entry
but may have
multiple exists



(c)



(d)

Outline

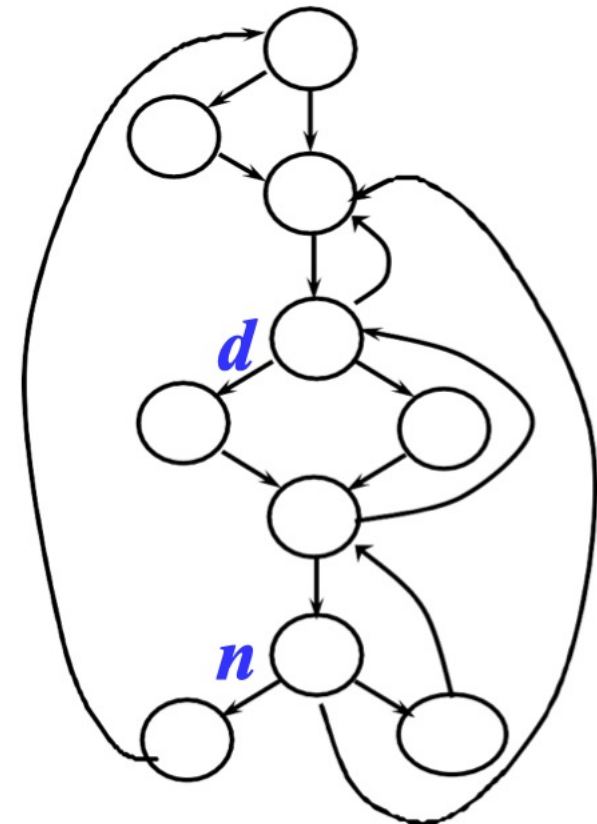
- Dominators
- Loop-Invariant Computations
- Induction Variables
- Array-Bounds Checks
- Loop Unrolling

Outline

- **Dominators**
- Loop-Invariant Computations
- Induction Variables
- Array-Bounds Checks
- Loop Unrolling

Dominator

- To optimize the loops, we must find them in the flow graph. The notion of ***dominators*** is useful for this purpose.
- **Dominator:** A node d dominates a node n if every path of directed edges from s_0 to n must go through d .
 - s_0 is the start node of the control-flow graph
- Every node dominates itself.
- n can have more than one dominators.



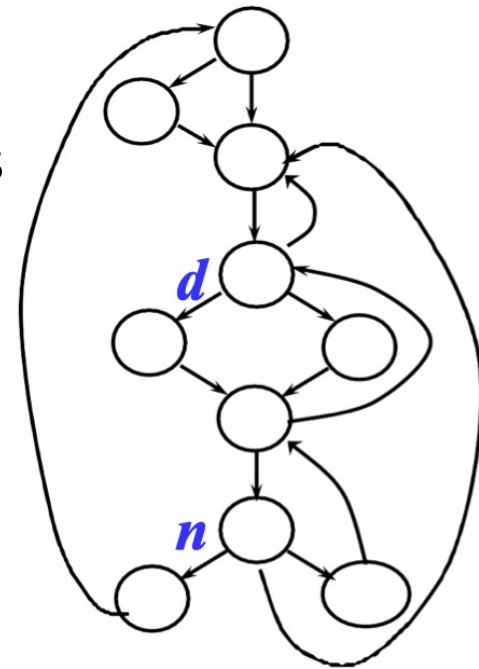
Finding Dominators

- **Algorithm for finding dominators:**

- let $D[n]$ be the set of nodes that dominate n , Then

$$D[s_0] = \{s_0\} \qquad D[n] = \{n\} \cup \left(\bigcap_{p \in \text{pred}[n]} D[p] \right) \qquad \text{for } n \neq s_0$$

- Initialize $D[n]$ (for $n \neq s_0$) to hold all nodes in the graph.
- Iteratively update $D[n]$ based on predecessors until you reach a fixed point



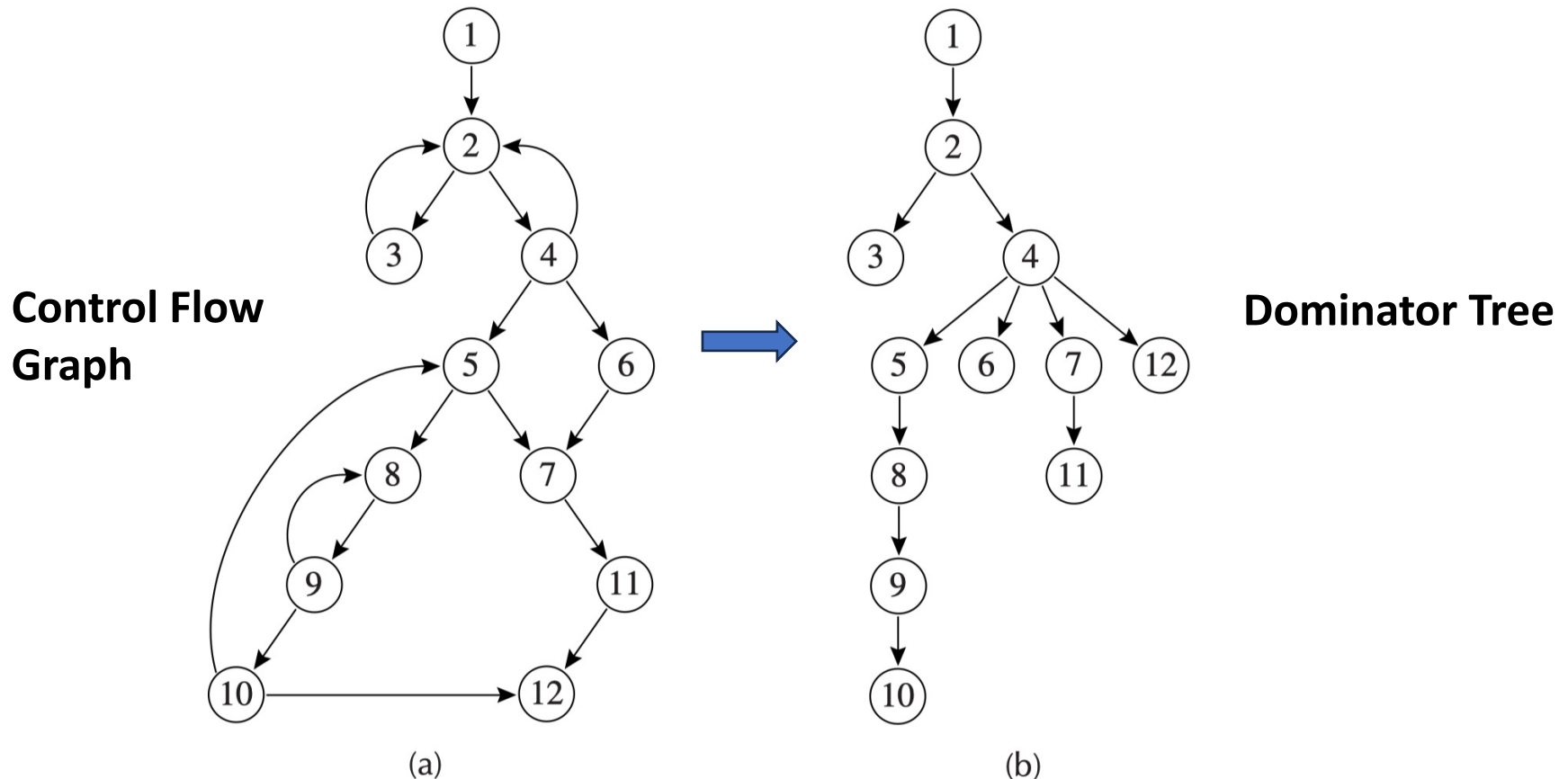
Immediate Dominators

- **Theorem:** In a connected graph, suppose d dominates n , and e dominates n . Then it must be that either d dominates e or e dominates d .
- Every node n (except s_0) has exactly one *immediate dominator*, $idom(n)$, such that:
 - $idom(n)$ is not the same node n
 - $idom(n)$ dominates n , and
 - $idom(n)$ does not dominate any other dominator of n

Dominator Tree

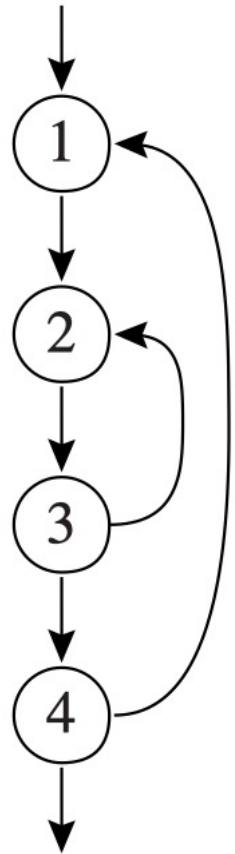
Dominator Tree:

- Draw a graph containing every node of the flow graph, and for every node n an edge from $idom(n)$ to n .
- The resulting graph will be a tree. Why?

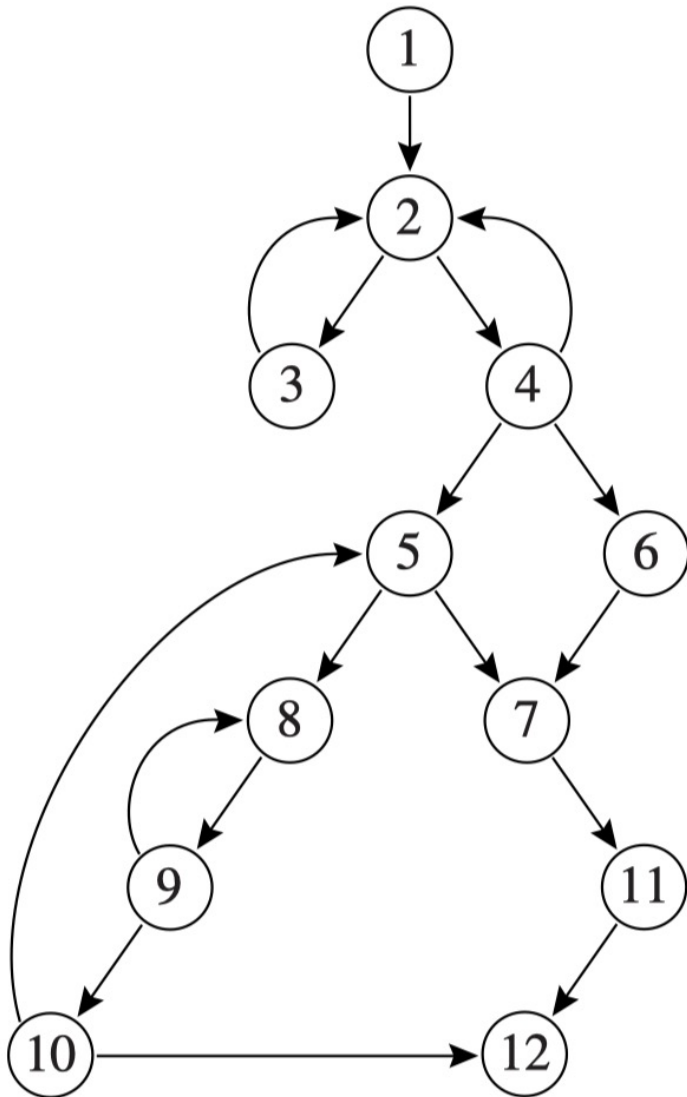


Natural Loops

- We care about whether a loop has a single entry node:
 - Yes => **certain initial conditions** to hold at the beginning of **each iteration** through the loop => ease the optimization
- This opportunity motivates the need for the definition of a natural loop.
- **Back Edge:** a flow-graph edge from a node n to a node h that dominates n is called a **back edge**.
- The **natural loop** of a back edge $n \rightarrow h$ is the set of nodes x such that:
 - h dominates x
 - there is a path from x to n not containing h .
- The header of this loop will be h



Natural Loops - Example

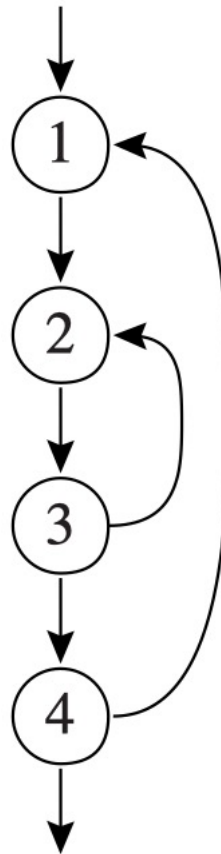


(a)

- The natural loop of the back edge **10 -> 5** includes nodes **5, 8, 9, 10**, and has the loop **8, 9** nested within it.
- A node ***h*** can be the header of more than one natural loop
 - $3 \rightarrow 2 \Rightarrow \{3, 2\}$
 - $4 \rightarrow 2 \Rightarrow \{4, 2\}$

Nested Loop

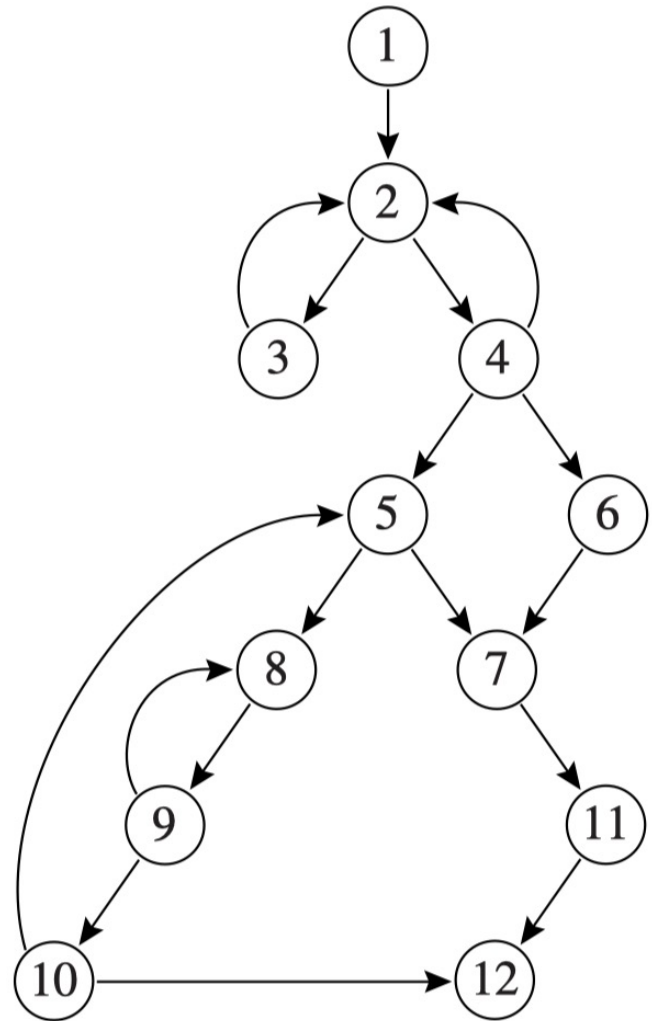
- **Nested Loops:** If Loops **A** and **B** are distinct headers and all nodes in **B** are also in **A** (i.e., $B \subseteq A$), then we say that loop **B** is **nested** within **A**, or that **B** is the **inner loop**.



Loop-Nest Tree

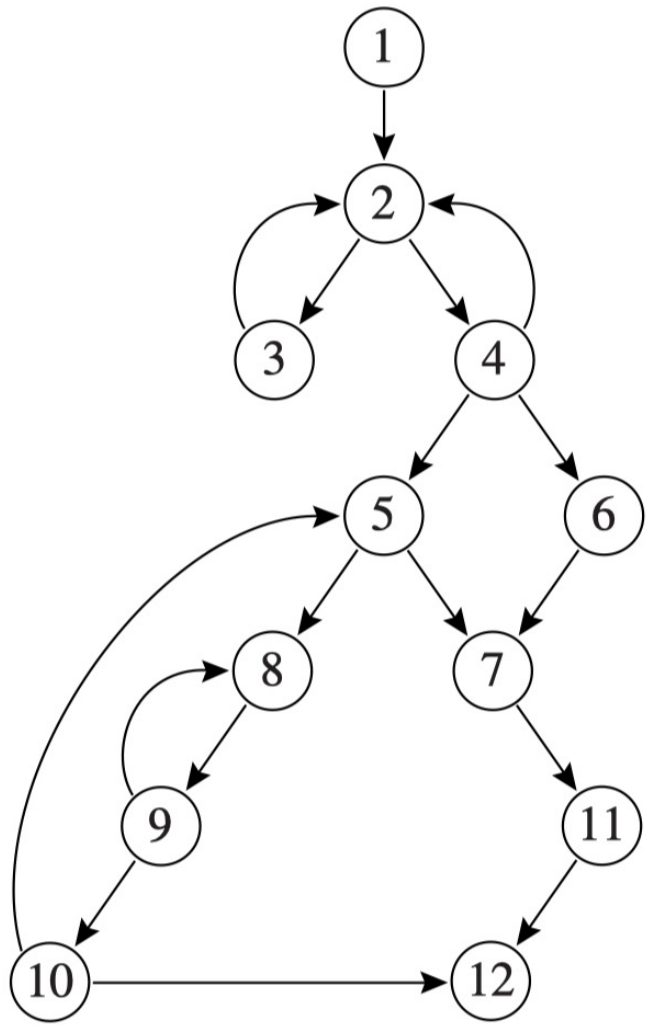
We can construct a **loop-nest tree** of loops in a program:

- Compute dominators of the flow graph G.
- Construct the dominator tree.
- Find all the natural loops, and thus all the loop-header nodes.
- For each loop header h , merge all the natural loops of h into a single loop, $\text{loop}[h]$.
- Construct the tree of loop headers (and implicitly loops), such that $h1$ is above $h2$ in the tree if $h2$ is in $\text{loop}[h1]$.

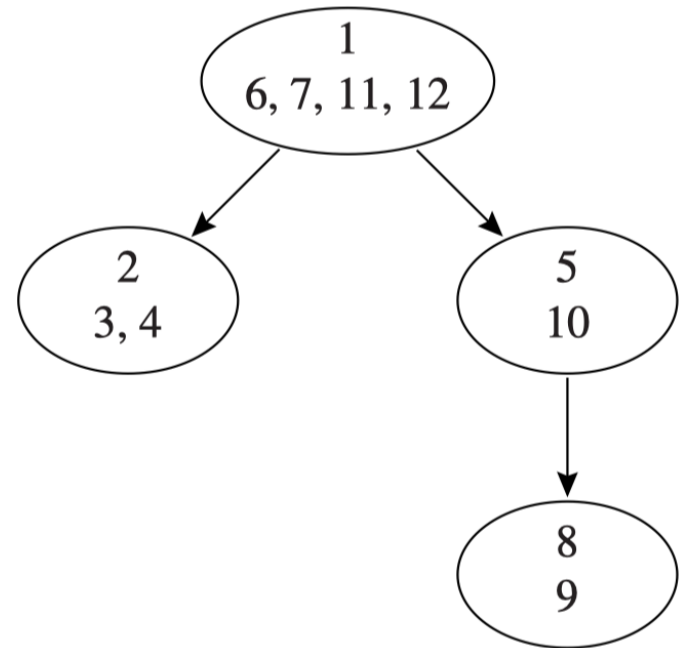


(a)

Loop-Nest Tree



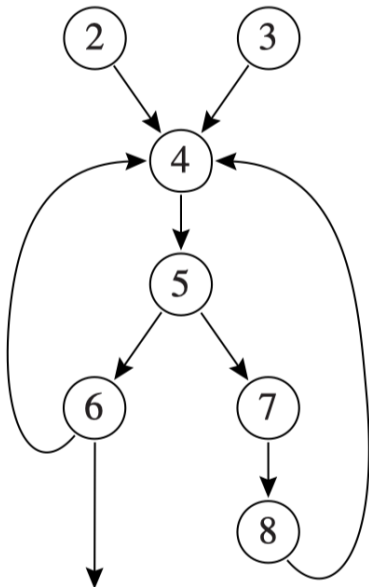
(a)



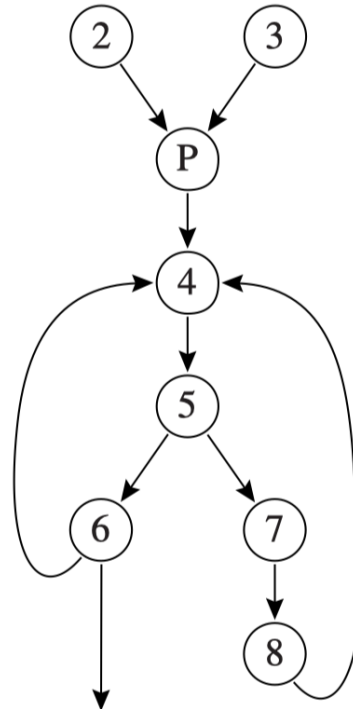
We could say that the entire procedure body is a pseudo-loop that sits at the root of the loop-nest tree.

Loop Preheader

- Many loop optimizations will insert statements immediately before the loop executes.
 - loop-invariant hoisting*: moves a statement (e.g., $a=1$) from inside the loop to immediately before the loop
- How to insert?



(a)



(b)

- Insert a new, initially empty, **preheader** node p
- all $y \rightarrow h$ from nodes y **outside the loop** are redirected to point to p

Outline

- Dominators
- **Loop-Invariant Computations**
- Induction Variables
- Array-Bounds Checks
- Loop Unrolling

Loop-Invariant Computations

- If a loop contains a statement $t \leftarrow a \oplus b$ such that a has the **same value** each time around the loop, and b has the **same value** each time, then t will also have the **same value** each time.
- We would like to **hoist** the computation out of the loop.
- How to know whether a have the same value every time?
- The definition $d: t \leftarrow a_1 \oplus a_2$ is **loop-invariant** within loop L if, for each operand a_i
 - a_i is a constant
 - or all the definitions of a_i that reach d are outside the loop
 - or only one definition of a_i reaches d , and that definition is loop-invariant

Iterative algorithm

Hoisting

- Suppose $t \leftarrow a \oplus b$ is loop-invariant. Can we hoist it out of the loop?

L_0 $t \leftarrow 0$ L_1 $i \leftarrow i + 1$ $t \leftarrow a \oplus b$ $M[i] \leftarrow t$ if $i < N$ goto L_1 L_2 $x \leftarrow t$	L_0 $t \leftarrow 0$ L_1 if $i \geq N$ goto L_2 $i \leftarrow i + 1$ $t \leftarrow a \oplus b$ $M[i] \leftarrow t$ goto L_1 L_2 $x \leftarrow t$	L_0 $t \leftarrow 0$ L_1 $i \leftarrow i + 1$ $t \leftarrow a \oplus b$ $M[i] \leftarrow t$ $t \leftarrow 0$ $M[j] \leftarrow t$ if $i < N$ goto L_1 L_2 $x \leftarrow t$	L_0 $t \leftarrow 0$ L_1 $M[j] \leftarrow t$ $i \leftarrow i + 1$ $t \leftarrow a \oplus b$ $M[i] \leftarrow t$ if $i < N$ goto L_1 L_2 $x \leftarrow t$
correct, faster	incorrect, not always execute the def	incorrect, more than one def of t	incorrect, a use of t before the def

Hoisting

The **criteria** for hoisting $d : t \leftarrow a \oplus b$ to the end of the loop preheader:

- d dominates all loop exits at which t is live-out
- and there is only one definition of t in the loop
- and t is not live-out of the loop preheader

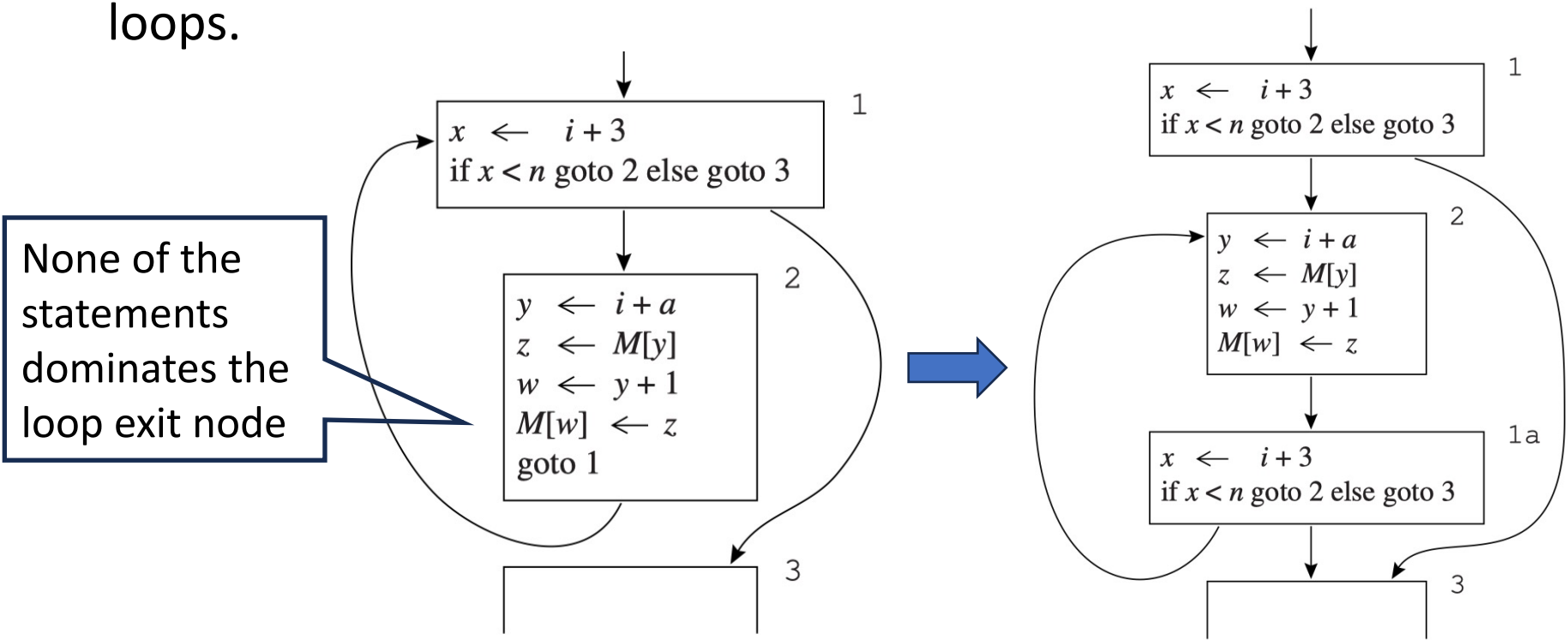
L_0 $t \leftarrow 0$ L_1 $i \leftarrow i + 1$ $t \leftarrow a \oplus b$ $M[i] \leftarrow t$ if $i < N$ goto L_1 L_2 $x \leftarrow t$	L_0 $t \leftarrow 0$ L_1 if $i \geq N$ goto L_2 $i \leftarrow i + 1$ $t \leftarrow a \oplus b$ $M[i] \leftarrow t$ goto L_1 L_2 $x \leftarrow t$	L_0 $t \leftarrow 0$ L_1 $i \leftarrow i + 1$ $t \leftarrow a \oplus b$ $M[i] \leftarrow t$ $t \leftarrow 0$ $M[j] \leftarrow t$ if $i < N$ goto L_1 L_2 $x \leftarrow t$	L_0 $t \leftarrow 0$ L_1 $M[j] \leftarrow t$ $i \leftarrow i + 1$ $t \leftarrow a \oplus b$ $M[i] \leftarrow t$ if $i < N$ goto L_1 L_2 $x \leftarrow t$
(a) Hoist	(b) Don't	(c) Don't	(d) Don't

Hoisting

Implicit side effects. These rules need modification if $t \leftarrow a \oplus b$ could raise some sort of arithmetic exception or have other side effects.

Turning while loops into repeat-until loops.

- “ d dominates all loop exits at which t is live-out”: tends to prevent many computations from being hoisted from **while** loops.



Outline

- Dominators
- Loop-Invariant Computations
- **Induction Variables**
- Array-Bounds Checks
- Loop Unrolling

Induction Variables

- Some loops have
 - a variable i that is incremented or decremented
 - a variable $j = i \cdot c + d$ where c and d are loop-invariant.
- We can calculate j 's value without reference to i
 - whenever i is incremented by a , $j = j + a \cdot c$

```

    s ← 0
    i ← 0
L1 : if i ≥ n goto L2
      j ← i · 4
      k ← j + a
      x ← M[k]
      s ← s + x
      i ← i + 1
      goto L1
L2
(a) Before
```

- i is a basic induction variable
- j and k are derived induction variable in the family of i

$$j = a_j + i \cdot b_j \quad (a_j=0, b_j=4)$$

$$(i, a_j, b_j) \Rightarrow (i, 0, 4)$$

$$k \leftarrow j + a \quad (a \text{ is loop-invariant})$$

$$k = i \cdot 4 + a = a + i \cdot 4$$

$$(i, a, 4)$$

Linear Induction Variable

- **Linear induction variable:** an induction variable changes by the same (constant or loop-invariant) amount in every iteration of the loop

```
     $s \leftarrow 0$   
 $L_1$  : if  $s > 0$  goto  $L_2$   
       $i \leftarrow i + b$   
       $j \leftarrow i \cdot 4$   
       $x \leftarrow M[j]$   
       $s \leftarrow s - x$   
      goto  $L_1$   
 $L_2$  :  $i \leftarrow i + 1$   
       $s \leftarrow s + j$   
      if  $i < n$  goto  $L_1$ 
```

(a) Before

- The basic induction variable i is incremented by different amounts in different iterations
 - not linear
- In some iterations $j = i \cdot 4$ and in other iterations the derived induction variable j gets (temporarily) left behind as i is incremented.

Induction Variables

```
     $s \leftarrow 0$   
     $i \leftarrow 0$   
 $L_1$  : if  $i \geq n$  goto  $L_2$   
        $j \leftarrow i \cdot 4$   
        $k \leftarrow j + a$   
        $x \leftarrow M[k]$   
        $s \leftarrow s + x$   
        $i \leftarrow i + 1$   
       goto  $L_1$   
 $L_2$ 
```

(a) Before

- induction-variable analysis
 - i & j
 - strength reduction
 - replace $i \cdot 4$
- 
- induction-variable elimination
 - $i \geq n \rightarrow k \geq 4n + a$
 - copy propagation

```
     $s \leftarrow 0$   
     $k' \leftarrow a$   
     $b \leftarrow n \cdot 4$   
     $c \leftarrow a + b$   
 $L_1$  : if  $k' \geq c$  goto  $L_2$   
        $x \leftarrow M[k']$   
        $s \leftarrow s + x$   
        $k' \leftarrow k' + 4$   
       goto  $L_1$   
 $L_2$ 
```

(b) After

Detection of Induction Variables

- **Basic induction variables:** The variable i is a basic induction variable in a loop L with header node h if the only definitions of i within L are of the form $i \leftarrow i + c$ or $i \leftarrow i - c$, where c is loop-invariant.
- **Derived induction variables:** The variable k is a derived induction variable in loop L , if
 1. There is only one def of k within L , of the form $k \leftarrow j \cdot c$ or $k \leftarrow j + d$, where j is an induction variable and c, d are loop-invariant
 2. **and** if j is derived induction variable in the family of i , then:
 - 2.1 the only def of j that reaches k is the one in the loop
 - 2.2 **and** there is no def of i on any path between the def of j and the def of k
Because we will convert k into a tuple related to i

Strength Reduction

- Multiplication is more expensive than addition.
- We would like to replace each derived induction variable defined as the form $j \leftarrow i \cdot c$ with an addition.
- For each derived induction variable j whose triple is (i, a, b) , i.e., $j \leftarrow a + i \cdot b$
 - make a new variable j'
 - After each assignment $i \leftarrow i + c$, make an assignment $j' \leftarrow j' + c \cdot b$, where $c \cdot b$ is a loop-invariant expression that may be computed in the loop preheader.
 - If c and b are both constant, then the multiplication may be done at compile time.
 - Replace the (unique) assignment to j with $j \leftarrow j'$
 - Initialize j' with $j' \leftarrow a + i \cdot b$ at the end of the loop preheader.

perform
dead-code
elimination

Strength Reduction - Example

```
s ← 0
i ← 0
L1: if i ≥ n goto L2
    j ← i · 4
    k ← j + a
    x ← M[k]
    s ← s + x
    i ← i + 1
    goto L1
L2:
```



```
s ← 0
i ← 0
L1: if i ≥ n goto L2
    j ← i · 4
    k ← j + a
    x ← M[k]
    s ← s + x
    i ← i + 1
    j' ← j' + 4
    goto L1
L2:
```

- make a new variable j'
- After each assignment $i \leftarrow i + c$, make an assignment $j' \leftarrow j' + c \cdot b$, where $c \cdot b$ is a loop-invariant expression that may be computed in the loop preheader. ($c = 1$, $b=4$)

Strength Reduction - Example

```
s ← 0
i ← 0
L1: if i ≥ n goto L2
    j ← i · 4
    k ← j + a
    x ← M[k]
    s ← s + x
    i ← i + 1
    j' ← j' + 4
    goto L1
L2:
```



```
s ← 0
i ← 0
j' ← 0
L1: if i ≥ n goto L2
    j ← j'
    k ← j + a
    x ← M[k]
    s ← s + x
    i ← i + 1
    j' ← j' + 4
    goto L1
L2:
```

- Replace the (unique) assignment to j with $j \leftarrow j'$
- Initialize j' with $j' \leftarrow a + i \cdot b$ at the end of the loop preheader.

Strength Reduction - Example

```
s ← 0
i ← 0
j' ← 0
L1: if i ≥ n goto L2
    j ← j'
    k ← j + a
    x ← M[k]
    s ← s + x
    i ← i + 1
    j' ← j' + 4
    goto L1
L2:
```



```
s ← 0
i ← 0
j' ← 0
k' ← a
L1: if i ≥ n goto L2
    j ← j'
    k ← k'
    x ← M[k]
    s ← s + x
    i ← i + 1
    j' ← j' + 4
    k' ← k' + 4
    goto L1
L2:
```

- Perform strength reduction on k.

Elimination

- After strength reduction
 - some of the induction variables are **not used** at all in the loop
 - and others are **used only in comparisons** with loop-invariant variables
- These induction variables can be **deleted**.
- A variable is **useless** in a loop **L** if
 - it is dead at all exits from **L**
 - and its only use is in a definition of itself
- All definitions of a useless variable may be **deleted**.

Can be deleted

```
s ← 0
i ← 0
j' ← 0
k' ← a
L1: if i ≥ n goto L2
    j ← j'
    k ← k'
    x ← M[k]
    s ← s + x
    i ← i + 1
    j' ← j' + 4
    k' ← k' + 4
    goto L1
L2:
```

Elimination

- After strength reduction
 - some of the induction variables are **not used** at all in the loop
 - and others are **used only in comparisons** with loop-invariant variables
- These induction variables can be **deleted**.
- A variable is **useless** in a loop **L** if
 - it is dead at all exits from **L**
 - and its only use is in a definition of itself
- All definitions of a useless variable may be **deleted**.

Can be deleted

```
s ← 0
i ← 0
j' ← 0
k' ← a
L1: if i ≥ n goto L2
      j ← j'
      k ← k'
      x ← M[k]
      s ← s + x
      i ← i + 1
      j' ← j' + 4
      k' ← k' + 4
      goto L1
L2:
```

Elimination

- After strength reduction
 - some of the induction variables are **not used** at all in the loop
 - and others are **used only in comparisons** with loop-invariant variables
- These induction variables can be **deleted**.
- A variable is **useless** in a loop L if
 - it is dead at all exits from L
 - and its only use is in a definition of itself
- All definitions of a useless variable may be **deleted**.

Can be deleted

```
s ← 0
i ← 0
j' ← 0
k' ← a
L1: if i ≥ n goto L2
j ← j'
k ← k'
x ← M[k]
s ← s + x
i ← i + 1
j' ← j' + 4
k' ← k' + 4
goto L1
L2:
```

Rewriting Comparisons

- A variable k is **almost useless**
 - If it is used only in comparisons against loop-invariant values and in definitions of itself,
 - and there is some other induction variable c in the same family that is not useless.
- An almost-useless variable $\Rightarrow$ a useless variable
 - by modifying the comparison to use c
- Example

Rewriting Comparisons - Example

```
s ← 0
i ← 0
k' ← a
L1: if i ≥ n goto L2
    k ← k'
    x ← M[k]
    s ← s + x
    i ← i + 1
    k' ← k' + 4
    goto L1
L2:
```



```
s ← 0
i ← 0
k' ← a
b ← n · 4
c ← a + b
L1: if k' ≥ c goto L2
    k ← k'
    x ← M[k]
    s ← s + x
    k' ← k' + 4
    goto L1
L2:
```

- almost useless variable: *i*
- related induction variable: *k'*
- $i \geq n \rightarrow k' \geq a + 4 \cdot n$
- $a + 4 \cdot n$ is loop-invariant, should be hoisted
- delete *i*

Rewriting Comparisons - Example

```
s ← 0
i ← 0
k' ← a
b ← n · 4
c ← a + b
L1: if k' > c goto L2
    k ← k'
    x ← M[k]
    s ← s + x
    k' ← k' + 4
    goto L1
L2:
```

copy
propagation



```
s ← 0
i ← 0
k' ← a
b ← n · 4
c ← a + b
L1: if k' > c goto L2
    x ← M[k']
    s ← s + x
    k' ← k' + 4
    goto L1
L2:
```

Induction Variables

```
     $s \leftarrow 0$   
     $i \leftarrow 0$   
 $L_1$  : if  $i \geq n$  goto  $L_2$   
        $j \leftarrow i \cdot 4$   
        $k \leftarrow j + a$   
        $x \leftarrow M[k]$   
        $s \leftarrow s + x$   
        $i \leftarrow i + 1$   
       goto  $L_1$   
 $L_2$ 
```

(a) Before

- induction-variable analysis
 - i & j
 - strength reduction
 - replace $i \cdot 4$
- 
- induction-variable elimination
 - $i \geq n \rightarrow k \geq 4n + a$
 - copy propagation

```
     $s \leftarrow 0$   
     $k' \leftarrow a$   
     $b \leftarrow n \cdot 4$   
     $c \leftarrow a + b$   
 $L_1$  : if  $k' \geq c$  goto  $L_2$   
        $x \leftarrow M[k']$   
        $s \leftarrow s + x$   
        $k' \leftarrow k' + 4$   
       goto  $L_1$   
 $L_2$ 
```

(b) After

Outline

- Dominators
- Loop-Invariant Computations
- Induction Variables
- **Array-Bounds Checks**
- Loop Unrolling

Array-Bounds Checks

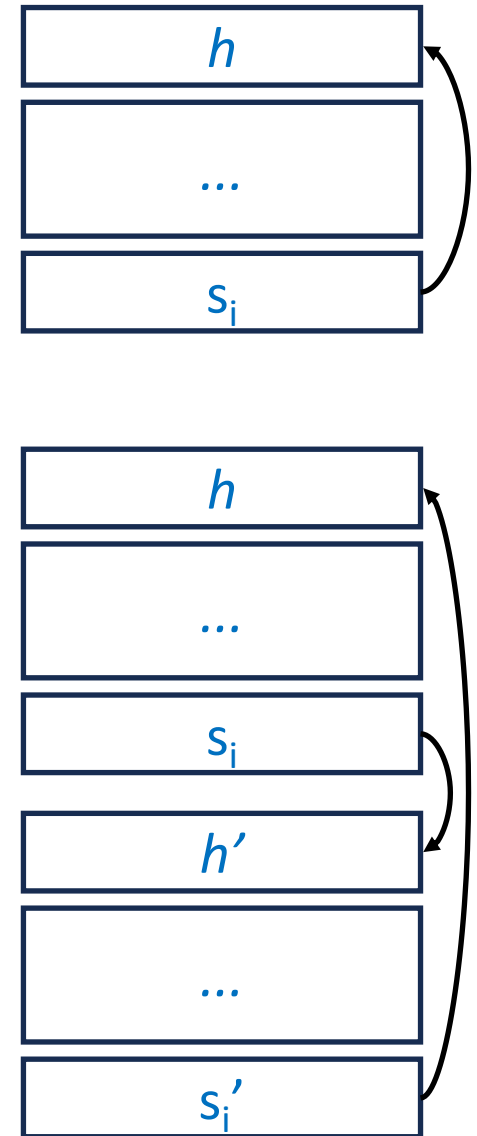
- Some programming languages automatically insert array-bounds checks on every subscript operation.
- We want the compiler to remove any checks that it can prove are redundant.
 - Removing all the redundant bounds checks is not computable
- Many array subscripts are of the form $a[i]$, where i is an induction variable.
 - The compiler can often understand well enough to optimize.
- The bounds for an array are generally of the form $0 \leq i \wedge i < N$.
- The criteria for eliminating a bounds check from a loop L are actually quite complicated.
 - Refer to the textbook.

Outline

- Dominators
- Loop-Invariant Computations
- Induction Variables
- Array-Bounds Checks
- **Loop Unrolling**

Loop Unrolling

- Some loops have such a small body that most of the time is spent incrementing the loop-counter variable and testing the loop-exit condition.
- **Unrolling**: putting two or more copies of the loop body in a row.
- Given a loop L with header node h and back edges $s_i \rightarrow h$, we can unroll the loop as follows:
 - Copy the nodes to make a loop L' with header h' and back edges $s_i' \rightarrow h'$
 - Change all the back edges in L from $s_i \rightarrow h$ to $s_i \rightarrow h'$
 - Change all the back edges in L' from $s_i' \rightarrow h'$ to $s_i' \rightarrow h$



Loop Unrolling

```
L1: x ← M[i]
    s ← s + x
    i ← i + 4
    if i < n goto L1 else L2
L2:
```

```
L1: x ← M[i]
    s ← s + x
    i ← i + 4
    if i < n goto L1' else L2
L1': x ← M[i]
    s ← s + x
    i ← i + 4
    if i < n goto L1 else L2
L2:
```

- Nothing useful has been accomplished, each “original” iteration still has an increment and a conditional branch.

Loop Unrolling

- We need an induction variable such that every increment $i \leftarrow i + \Delta$ dominates every back edge of the loop
- Agglomerate the increments and loop-exit tests
 - can only handle even times of iter

```
L1: x ← M[i]
    s ← s + x
    i ← i + 4
    if i < n goto L1' else L2
L1': x ← M[i]
    s ← s + x
    i ← i + 4
    if i < n goto L1 else L2
L2:
```



```
L1: x ← M[i]
    s ← s + x
    x ← M[i+4]
    s ← s + x
    i ← i + 8
    if i < n goto L1 else L2
L2:
```

Fragile

Loop Unrolling

- We need an induction variable such that every increment $i \leftarrow i + \Delta$ dominates every back edge of the loop
- Agglomerate the increments and loop-exit tests => Fragile
 - can only handle even times of iter
- Execute “odd” iterations in an epilogue => Robust

```
L1: x ← M[i]
    s ← s + x
    x ← M[i+4]
    s ← s + x
    i ← i + 8
    if i < n goto L1 else L2
L2:
```



```
    if i < n - 8 goto L1 else L2
L1: x ← M[i]
    s ← s + x
    x ← M[i+4]
    s ← s + x
    i ← i + 8
    if i < n goto L1 else L3
L2: x ← M[i]
    s ← s + x
    i ← i + 4
    if i < n goto L2 else L3
L3
```

Fragile

Robust

Summary

- Dominators
- Loop-Invariant Computations
- Induction Variables
- Array-Bounds Checks
- Loop Unrolling

Final Examination

- 2025年6月19日 (10:30-12:30) 玉泉教7-206
 - **以教务系统为准**
- 半开卷 (只允许带三张A4纸, 打印或手写, 可正反面)
- 考试范围: 第1-11章, 第13-14章, 第18章
- 题型: 判断题、选择题、问答题
- 补充说明: 第18章只考1, 2两个小节, 且只出判断、选择题。
- **提示:**
 - 题量可能较大, 熟练度要高
 - 每一章的重要概念、重要算法一定要弄懂
 - 作业题、quiz题目要会做
- **后续信息如有更新会在钉钉群发布, 请大家注意关注。**

重要内容

- 每一章的基本概念：
 - 例如，编译器有哪些阶段？每个阶段都在干什么？什么是first set？什么是符号表？什么是garbage？等等
- 编译器的每个阶段如何将输入转换为输出：
 - 每个阶段的输入是什么？输出是什么？
 - 会构建哪些数据结构，如何构建？通过哪些流程/算法将输入转换为输出的？不同算法的优缺点是什么？
 - 给你一个输入，你能手动构建出这个输出吗？
- 每一章节中介绍的算法：
 - 深入理解、灵活应用，明确不同算法的优缺点及适用场景
- 问题？

谢谢大家